đề bài

PHẦN 1: BÀI KIỂM TRA THỰC HÀNH LẬP TRÌNH NHÚNG STM32

1. Mục tiêu bài test

Đánh giá khả năng:

- Thiết kế firmware thời gian thực có tính **ổn định và tất định**
- Làm chủ **Timer, PWM, Interrupt, DMA**
- Viết code tối ưu, không gây nghẽn CPU
- Tổ chức code rõ ràng, dễ mở rộng

---

2. Mô tả bài toán

Viết chương trình C/C++ trên STM32 để điều khiển 3 LED (A, B, C) tạo hiệu ứng ánh sáng dạng sóng:

- Độ sáng của mỗi LED phải thay đổi **mượt mà, liên tục**
- 3 LED phải thay đổi độ sáng **tuần hoàn và lệch pha đều nhau**
- Hiệu ứng phải chạy ổn định theo thời gian, **không bị giật, không chớp bất thường**

---

3. Yêu cầu kỹ thuật bắt buộc

3.1. Sử dụng ngoại vi

- Chỉ sử dụng **01 Timer duy nhất**
- Timer này phải:
    - Xuất **3 kênh PWM** điều khiển 3 LED
    - Đồng thời dùng **ngắt tràn (Update Interrupt)** để cập nhật dữ liệu

---

3.2. Tín hiệu PWM

- Tần số PWM ≥ **1kHz**
- Ưu tiên cấu hình **Center-Aligned**
- Duty cycle phải thay đổi liên tục theo thời gian (không bật/tắt đột ngột)

3.3. Thuật toán điều khiển độ sáng

- Phải sử dụng **mảng dữ liệu tĩnh (Lookup Table)**
- Mảng phải có tối thiểu **100 phần tử**
- 3 LED phải có độ lệch đều nhau theo chu kỳ của mảng

3.4. Cập nhật dữ liệu

- Việc cập nhật giá trị PWM phải thực hiện trong **Interrupt**
- Chu kỳ cập nhật: **1ms – 5ms**
- Không được sử dụng bất kỳ cơ chế delay/blocking nào

3.5. Ràng buộc hiệu năng (bắt buộc)

Trong toàn bộ chương trình:

- ❌ Không sử dụng `float` hoặc `double` trong Interrupt
- ❌ Không gọi hàm toán học (ví dụ `sin()`) trong runtime
- ❌ Không sử dụng `HAL_Delay()` hoặc bất kỳ delay blocking nào
- ❌ Không để xảy ra hiện tượng treo, giật, hoặc mất ổn định
- ✅ Code trong Interrupt phải **ngắn gọn, tối ưu, dễ kiểm soát thời gian thực thi**

---

4. Yêu cầu về hành vi hệ thống (tiêu chí chấm trực tiếp)

Ứng viên sẽ được đánh giá dựa trên kết quả chạy thực tế:

4.1. Tính mượt

- Độ sáng thay đổi liên tục, không có bước nhảy rõ rệt

4.2. Tính ổn định

- Hệ thống chạy lâu không bị:
  - giật
  - lệch pha
  - sai chu kỳ

4.3. Tính đồng đều

- 3 LED phải duy trì khoảng lệch đều nhau trong toàn bộ thời gian chạy

4.4. Tính nhất quán thời gian

- Chu kỳ lặp lại phải đều, không bị nhanh/chậm bất thường

---

5. Yêu cầu về mã nguồn

5.1. Tổ chức code

- Phân tách rõ:
  - phần khởi tạo (init)
  - phần xử lý runtime
- Không viết logic phức tạp trong `main()` loop

5.2. Xử lý Interrupt

- Code trong ISR phải:
  - ngắn gọn
  - không chứa xử lý dư thừa
- Quản lý chỉ số mảng an toàn (không out-of-bounds)

5.3. Biến và dữ liệu

- Các biến dùng trong Interrupt phải khai báo phù hợp (ví dụ `volatile` nếu cần)
- Không sử dụng bộ nhớ động

---

6. Yêu cầu nâng cao (không bắt buộc)

6.1. Điều khiển tốc độ

- Có thể thay đổi tốc độ hiệu ứng thông qua một biến (`speed`)
- Việc thay đổi không làm mất tính đồng đều giữa các LED

6.2. Tương tác

- Sử dụng 1 nút nhấn để:
  - Bật/Tắt hiệu ứng

6.3. Tối ưu bằng DMA

- Không sử dụng Interrupt để cập nhật PWM
- Dùng DMA ở chế độ Circular để tự động cập nhật dữ liệu

---

7. Tiêu chí đánh giá

Ứng viên sẽ được đánh giá dựa trên:

- ✅ Kết quả chạy thực tế (ưu tiên cao nhất)
- ✅ Độ ổn định theo thời gian
- ✅ Cách sử dụng ngoại vi
- ✅ Mức độ tối ưu trong Interrupt
- ✅ Tổ chức và chất lượng mã nguồn

✅ Tóm tắt tinh thần bài test

Đây là bài test thực hành thuần túy.
Ứng viên không cần giải thích lý thuyết.
Kết quả được đánh giá dựa trên **hành vi hệ thống và chất lượng code**.

Hướng dẫn

PHẦN 2: HƯỚNG DẪN & CODE THAM KHẢO (BASELINE)

1. Mục tiêu phần này

- Cung cấp một phiên bản **chạy được**
- Nhưng **chưa tối ưu hoàn toàn**
- Ứng viên cần:
    - Refactor
    - Tối ưu ISR
    - Cải thiện cấu trúc
    - Đảm bảo timing tốt hơn

---

2. Ý tưởng triển khai cơ bản

- 1 Timer:
    - PWM: CH1, CH2, CH3
    - Update Interrupt: 1ms
- LUT: 120 điểm
- 3 index lệch pha:
    - A = 0
    - B = 40
    - C = 80

---

3. Code tham khảo (STM32 HAL)

3.1. Sine Lookup Table

```
#define TABLE_SIZE 120
#define PWM_MAX    999

const uint16_t sine_table[TABLE_SIZE] = {
    500,526,552,578,603,628,652,675,698,719,
    740,759,777,794,809,823,835,846,855,862,
    868,872,874,875,874,872,868,862,855,846,
    835,823,809,794,777,759,740,719,698,675,
```

```
    652,628,603,578,552,526,500,473,447,421,
    396,371,347,324,301,280,259,240,222,205,
    190,176,164,153,144,137,131,127,125,124,
    125,127,131,137,144,153,164,176,190,205,
    222,240,259,280,301,324,347,371,396,421,
    447,473,500,526,552,578,603,628,652,675,
    698,719,740,759,777,794,809,823,835,846
};
```

3.2. Biến toàn cục

```
volatile uint16_t idx_A = 0;
volatile uint16_t idx_B = 40;
volatile uint16_t idx_C = 80;

volatile uint8_t run = 1;
```

3.3. Khởi động PWM + Interrupt

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);

HAL_TIM_Base_Start_IT(&htim1);
```

3.4. ISR (Baseline – CHƯA TỐI ƯU)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM1)
    {
        if(run)
        {
            __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, sine_table[idx_A]);
            __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_2, sine_table[idx_B]);
            __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_3, sine_table[idx_C]);

            idx_A = (idx_A + 1) % TABLE_SIZE;
            idx_B = (idx_B + 1) % TABLE_SIZE;
            idx_C = (idx_C + 1) % TABLE_SIZE;
        }
```

```
    }
}
```

3.5. Main loop

```
while (1)
{
    // intentionally empty
}
```

⚠️ 4. Các vấn đề cố tình tồn tại trong code

Ứng viên **PHẢI phát hiện và cải thiện**:

❌ 1. Dùng toán tử % trong ISR

```
idx_A = (idx_A + 1) % TABLE_SIZE;
```

👉 Vấn đề:

- Tốn CPU hơn cần thiết
- Không phù hợp ISR tối ưu

❌ 2. Dùng macro HAL trong ISR

```
__HAL_TIM_SET_COMPARE(...)
```

👉 Vấn đề:

- Có overhead
- Không tối ưu timing

❌ 3. ISR chưa tối giản

- Có `if(run)`
- Có nhiều call function

👉 Có thể:

- tách logic ra ngoài
- giảm nhánh

❌ 4. Không kiểm soát timing ISR

- Không biết ISR chạy bao lâu
- Không đảm bảo < chu kỳ 1ms

❌ 5. LUT cố định, chưa scale theo ARR

- Hardcode 999

❌ 6. Không có speed control

---

5. Yêu cầu ứng viên cải thiện

Ứng viên cần nâng cấp code theo các hướng sau:

✅ 5.1. Tối ưu ISR

- Không dùng `%`
- Giảm branch
- Giảm function call

👉 Gợi ý:

```
idx_A++;
if(idx_A >= TABLE_SIZE) idx_A = 0;
```

✅ 5.2. Truy cập trực tiếp thanh ghi

Thay:

__HAL_TIM_SET_COMPARE(...)

Bằng:

TIM1->CCR1 = sine_table[idx_A];

✅ 5.3. Tối ưu cấu trúc

- Tách:
  - init
  - runtime
- ISR chỉ làm việc tối thiểu

✅ 5.4. Thêm speed control

Ví dụ:

volatile uint8_t step = 1;

idx_A += step;

✅ 5.5. Đảm bảo không out-of-bounds

✅ 5.6. Bonus: DMA

- Circular mode
- Trigger từ Timer Update

---

6. Tiêu chí đánh giá nâng cao

Ứng viên được đánh giá dựa trên:

🔹 Mức 1 (Pass cơ bản)

- Chạy đúng
- LED mượt
- Không crash

🔹 Mức 2 (Tốt)

- ISR tối ưu
- Không dùng %
- Code rõ ràng

🔹 Mức 3 (Rất tốt)

- Truy cập register trực tiếp
- Có speed control
- Cấu trúc clean

🔹 Mức 4 (Xuất sắc)

- Dùng DMA
- CPU gần như idle
- Timing cực ổn định

🔥 Tóm tắt tinh thần phần 2

Đây không phải code mẫu để copy.
Đây là code “chưa đủ tốt”.
Ứng viên phải chứng minh họ có thể làm tốt hơn.

Mã nguồn

🧩 main.c – FULL CODE

1. Include & define

```
#include "main.h"

#define TABLE_SIZE 120
#define PWM_MAX    999
```

---

2. Sine Lookup Table

```
const uint16_t sine_table[TABLE_SIZE] = {
    500,526,552,578,603,628,652,675,698,719,
    740,759,777,794,809,823,835,846,855,862,
    868,872,874,875,874,872,868,862,855,846,
    835,823,809,794,777,759,740,719,698,675,
    652,628,603,578,552,526,500,473,447,421,
    396,371,347,324,301,280,259,240,222,205,
    190,176,164,153,144,137,131,127,125,124,
    125,127,131,137,144,153,164,176,190,205,
    222,240,259,280,301,324,347,371,396,421,
    447,473,500,526,552,578,603,628,652,675,
    698,719,740,759,777,794,809,823,835,846
};
```

---

3. Biến toàn cục

```
TIM_HandleTypeDef htim1;

volatile uint16_t idx_A = 0;
volatile uint16_t idx_B = 40;
volatile uint16_t idx_C = 80;

volatile uint8_t run = 1;
volatile uint8_t step = 1;
```

---

4. Function prototype

```
void SystemClock_Config(void);
```

```
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
```

---

5. Main function

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_TIM1_Init();

  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);

  HAL_TIM_Base_Start_IT(&htim1);

  while (1)
  {
    // Main loop rảnh
  }
}
```

⚡ 6. TIMER INIT (TIM1)

```
static void MX_TIM1_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 71;     // 72MHz / (71+1) = 1MHz
  htim1.Init.CounterMode = TIM_COUNTERMODE_CENTERALIGNED1;
  htim1.Init.Period = PWM_MAX;   // 1kHz PWM
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

  HAL_TIM_Base_Init(&htim1);
  HAL_TIM_PWM_Init(&htim1);
```

```
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0;
    sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

    HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1);
    HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2);
    HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_3);

    sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
    HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig);
}
```

🔁 7. ISR – Update PWM

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM1)
    {
        if(run)
        {
            // Update PWM duty
            TIM1->CCR1 = sine_table[idx_A];
            TIM1->CCR2 = sine_table[idx_B];
            TIM1->CCR3 = sine_table[idx_C];

            // Update index
            idx_A += step;
            if(idx_A >= TABLE_SIZE) idx_A -= TABLE_SIZE;

            idx_B += step;
            if(idx_B >= TABLE_SIZE) idx_B -= TABLE_SIZE;

            idx_C += step;
            if(idx_C >= TABLE_SIZE) idx_C -= TABLE_SIZE;
        }
    }
}
```

🔘 8. GPIO INIT (LED pins)

```
static void MX_GPIO_Init(void)
{
    __HAL_RCC_GPIOA_CLK_ENABLE();

    GPIO_InitTypeDef GPIO_InitStruct = {0};

    // TIM1 CH1, CH2, CH3 (PA8, PA9, PA10 ví dụ)
    GPIO_InitStruct.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;

    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

⏱ 9. System Clock (ví dụ 72MHz)

```
void SystemClock_Config(void)
{
    // Dùng config mặc định từ CubeMX
}
```

✅ 10. Kết quả khi chạy

- 3 LED:
  - Sáng/tắt **mượt**
  - Lệch pha đều
  - Không giật
- Main loop:
  - Idle hoàn toàn
- ISR:
  - Nhẹ, ổn định

⚠️ 11. Những điểm vẫn có thể cải thiện (cố ý giữ lại)

Ứng viên giỏi sẽ nhận ra:

- Chưa dùng DMA
- Chưa đo thời gian ISR
- Chưa tách layer driver
- LUT chưa generate dynamic
- Không có EXTI control

🔥 Tóm lại

👉 Code này:

- ✅ Chạy được ngay
- ✅ Đúng yêu cầu đề
- ⚠️ Chưa tối ưu hoàn toàn